LoRe: A Programming Model for Verifiably Safe Local-first Software

JULIAN HAAS and RAGNAR MOGK, Technische Universität Darmstadt, Germany
ELENA YANAKIEVA and ANNETTE BIENIUSA, Rheinland-Pfälzische Technische Universität Kaiserslautern-Landau, Germany
MIRA MEZINI, Technische Universität Darmstadt, Germany, and National Research Center for Applied Cybersecurity (ATHENE), Germany, and Hessian Center for Artificial Intelligence (hessian.AI), Germany

Local-first software manages and processes private data locally while still enabling collaboration between multiple parties connected via partially unreliable networks. Such software typically involves interactions with users and the execution environment (the outside world). The unpredictability of such interactions paired with their decentralized nature make reasoning about the correctness of local-first software a challenging endeavor. Yet, existing solutions to develop local-first software do not provide support for automated safety guarantees and instead expect developers to reason about concurrent interactions in an environment with unreliable network conditions.

We propose *LoRe*, a programming model and compiler that automatically verifies developer-supplied safety properties for local-first applications. *LoRe* combines the declarative data flow of reactive programming with static analysis and verification techniques to precisely determine concurrent interactions that violate safety invariants and to selectively employ strong consistency through coordination where required. We propose a formalized proof principle and demonstrate how to automate the process in a prototype implementation that outputs verified executable code. Our evaluation shows that *LoRe* simplifies the development of safe local-first software when compared to state-of-the-art approaches and that verification times are acceptable.

CCS Concepts: • **Software and its engineering** → **Formal software verification**; **Distributed programming languages**; *Data flow languages*; *Consistency;* • **Theory of computation** → *Pre- and post-conditions*; *Program specifications* • **Computer systems organization** → Peer-to-peer architectures;

Additional Key Words and Phrases: Local-first software, reactive programming, invariants, consistency, automatic verification

ACM Reference format:
Julian Haas, Ragnar Mogk, Elena Yanakieva, Annette Bieniusa, and Mira Mezini. 2023. LoRe: A Programming Model for Verifiably Safe Local-first Software. *ACM Trans. Program. Lang. Syst.* 46, 1, Article 2 (January 2023), 26 pages.
https://doi.org/10.1145/3633769

This work was funded by the German Federal Ministry of Education and Research together with the Hessen State Ministry for Higher Education (ATHENE), the German Research Foundation (SFB 1053), and the German Federal Ministry for Economic Affairs and Climate Action project SafeFBDC (01MK21002K).
Authors' addresses: J. Haas and R. Mogk, Technische Universität Darmstadt, Department of Computer Science, 64289 Darmstadt, Germany; e-mails: haas@cs.tu-darmstadt.de, mogk@cs.tu-darmstadt.de; E. Yanakieva and A. Bieniusa, Rheinland-Pfälzische Technische Universität Kaiserslautern-Landau, Department of Computer Science, 67663 Kaiserslautern, Germany; e-mails: yanakieva@cs.uni-kl.de, bieniusa@cs.uni-kl.de; M. Mezini, Technische Universität Darmstadt, Department of Computer Science, 64289 Darmstadt, Germany; e-mail: mezini@cs.tu-darmstadt.de.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
0164-0925/2024/01-ART2 $15.00
https://doi.org/10.1145/3633769

1 INTRODUCTION

Applications that enable multiple parties connected via partially unreliable networks to collaboratively process data prevail today. An illustrative example is a distributed calendar application with services to add or modify appointments, where a user may maintain multiple calendars on different devices, may share calendars with other users, back them up in a cloud; calendars must be accessible to users in a variety of scenarios, including offline periods, e.g., while traveling—yet, planning appointments may require coordination between multiple parties. The calendar application is representative for other collaborative data-driven software, such as group collaboration tools, digital (cross-organizational) supply chains, multiplayer online gaming, and more.

The dominating software architecture for such applications is centralized: Data is collected, managed, and processed centrally in data centers, while devices on the edge of the communication infrastructure serve primarily as interfaces to users and the outside world. This architecture simplifies the software running on edge devices, since concerns like consistent data changes to ensure safety properties are managed centrally. However, this comes with issues including loss of control over data ownership and privacy, insufficient offline availability, poor latency, inefficient use of communication infrastructure, and waste of (powerful) computing resources on the edge.

To address these issues, local-first principles for software development have been formulated [25], calling for moving data management and processing to edge devices instead of confining the data to clouds. But for programming approaches that implement these principles to be viable alternatives to the centralized approach, they must support automatically verifiable safety guarantees to counter for the simplifying assumptions afforded by a centralized approach. Unfortunately, existing approaches to programming local-first applications such as *Yjs* [42] or *Automerge* [4] do not provide such guarantees. They use ***conflict-free replicated data types (CRDTs)*** [47] to store the parts of their state that is shared across devices and rely on callbacks for modeling and managing state that changes in both time and space. CRDTs have been invented in the context of geo-replicated databases and are available as off-the-shelf databases [46] or libraries [16, 24]. However, the strongest consistency level ensured by CRDTs, causal consistency [34], is not enough to maintain invariants that require coordination among the participants, e.g., the invariant that employees should not enter more than the available vacation days in a use of the calendar app in a business setting. At the same time, we need to delimit the scope of coordination to "maximize combinations of availability and consistency that make sense for a particular application" [10]. To find the set of interactions that actually need coordination, in a local-first setting, one must reason about possible interleavings of their data flows "end-to-end," i.e., from the interface to the outside world, through device-local data-dependency chains, to remote devices and back. The unpredictability of the interactions triggered by the outside world, concurrently at different devices, paired with the absence of a central authority and the prevailing implicit dependencies in current callback-centered programming models, makes such reasoning without automated support a challenging, error-prone endeavour.

To close this gap, we propose a programming model for local-first applications that features explicit safety properties and automatically enforces them. The model has three core building blocks: *reactives*, *invariants*, and *interactions*. *Reactives* express values that change in time but also in space by being replicated over multiple devices. They enable systematic treatment of complex state, dependencies, and concurrent changes, enabling developers to reason in terms of functional composition. *Invariants* are formula in first-order logic specifying safety properties that must hold at all times when the application interacts with the outside world, or values of reactives are observable. *Interactions* interface to the outside world and encapsulate changes to all reactives affected by interactions with it (state directly changed by the interactions, device-local values derived from the changed state, and shared state at remote devices). They serve as language-managed

cross-device data flows that automatically use best-in-class consistency. On one device, interactions are logically processed instantaneously, i.e., no intermediate states are observable, similar to atomicity in databases. We use automatic verification with invariants as verification obligations to identify interactions that need coordination across devices, for which the compiler generates the coordination protocol; all other interactions become visible in causal order. This way, the compiler makes an application-specific availability-safety trade-off.

The availability-safety trade-off has been explored in the systems and database community under the term "coordination avoidance" and there exist approaches that leverage user-specified safety invariants to synthesize distributed objects or to correctly configure the consistency level for each database operation [7, 12, 18, 52]. Our work draws inspiration from these approaches, but differs in two key aspects: (1) Instead of geo-replicated databases, we target a peer-to-peer local-first setting with unique challenges; specifically, we do not assume any centralized authority and feature offline availability and interactions with the outside world. (2) Instead of programming applications against the interface to a distributed datastore/object, which fosters designs split into two separate tiers—an automatically managed monolithic data store and the application tier that uses the store's API—we provide a language-integrated mixed consistency with whole program guarantees, i.e., we verify the safety of derived data "all the way down." This is necessary in a local-first setting, where most of the complexity arises from the interactions with the external world and are handled as part of the application logic, not as part of the data store.

In summary, we make the following contributions:

(1) A programming model for local-first applications with verified safety properties (Section 2), called LoRe. While individual elements of the model, e.g., CRDTs or reactives, are not novel, they are repurposed, combined, and extended in a unique way to systematically address specific needs of local-first applications with regard to ensuring safety properties.
(2) A formal definition of the model including a formal notion of invariant preservation and confluence for interactions, and a modular verification that invariants are never violated. In case of potential invariant violations due to concurrent executions on multiple devices, LoRe automatically adds the necessary coordination logic (Section 3).
(3) A verifying compiler[1] that translates LoRe programs to Viper [39] for automated verification and to Scala for execution including the necessary coordination logic to guarantee the specified safety invariants (Section 4).
(4) An evaluation of LoRe in two case studies (Section 5). Our evaluation validates two claims we make about the programming model proposed: (a) It facilitates the development of safe local-first software, and (b) it enables an efficient and modular verification of safety properties. It further shows that the additional safety properties offered by our model do not come with prohibitive costs in terms of verification effort and time.

2 LoRe IN A NUTSHELL

We introduce the concepts of LoRe along the example of a distributed calendar for tracking work meetings and vacation days. LoRe is an external DSL that compiles to Scala (for execution) and Viper IR [39] (for verification); its syntax is inspired by both. A LoRe program defines a distributed application that runs on multiple physical or virtual devices.[2] Listing 1 shows a simplified implementation of the calendar example application in LoRe. As any LoRe program, it consists of

[1]The source code of our prototype implementation is available at https://github.com/stg-tud/LoRe
[2]We assume that every device is running the same application code (i.e., the same binary), and different types of devices (such as client and server) are modeled by limiting them to execute a subset of the defined interactions.

```
1  type Calendar = AWSet[Appointment]
2  val work: Source[Calendar] = Source(AWSet())
3  val vacation: Source[Calendar] = Source(AWSet())
4
5  val all_appointments: Derived[Set[Appointment]] = Derived{ work.toSet.union(
       vacation.toSet) }
6  val remaining_vacation: Derived[Int] = Derived{ 30 - sumDays(vacation.toSet) }
7
8  val add_appointment: Unit = Interaction[Calendar][Appointment]
9    .requires{ cal => a => get_start(a) < get_end(a) }
10   .requires{ cal => a => !(a in cal.toSet)}
11   .executes{ cal => a => cal.add(a) }
12   .ensures { cal => a => a in cal.toSet }
13 val add_vacation: Unit = add_appointment.modifies(vacation)
14   .requires{ cal => a => remaining_vacation - a.days >= 0}
15 val add_work: Unit = add_appointment.modifies(work)
16
17 UI.display(all_appointments, remaining_vacation)
18 UI.vacationDialog.onConfirm{a => add_vacation.apply(a)}
19
20 invariant forall a: Appointment ::
21   a in all_appointments ==> get_start(a) < get_end(a)
22
23 invariant remaining_vacation >= 0
```

Listing 1. Distributed calendar application.

replicated state (`Source` reactives in Lines 2 and 3), local values derived from them (`Derived` reactives in Lines 5 and 6), interactions (Lines 8–15), and invariants (Lines 20–23).

2.1 Reactives

Reactives are the composition units in a LoRe program. We distinguish two types of them: *source* and *derived* reactives, declared by the keywords `Source` and `Derived`, respectively. Source reactives are values that are directly changed through interactions. Their state is modeled as CRDTs [44, 47] and is replicated between the different devices collaborating on the application. Derived reactives represent local values that are automatically computed by the system from the values of other reactives (source or derived). Changes to source reactives automatically (a) trigger updates of derived reactives and (b) cause devices to asynchronously send update messages to the other devices, which then merge the changes into their local state. Together, local propagations and asynchronous cross-device update messages ensure that users always have a consistent view of the overall application state. All reactives are statically declared in the program source code. LoRe then statically extracts knowledge about the data flow for modular verification and to minimize the proof goals (cf. Section 4.1). We discuss the technical implications of static reactives in Section 7.

Listing 1 shows two source reactives, `work` and `vacation` (Lines 2 and 3), each modeling a calendar as a set of appointments. The work calendar tracks work meetings, while the vacation calendar contains registered vacation days. When defining a source reactive, programmers have to choose a CRDT for the reactive's internal state. LoRe offers a selection of pre-defined CRDTs including various standard data types such as sets, counters, registers and lists. Further data types can be supported by providing a Viper specification for that data type. In this case, an *add-wins-set* (a set CRDT where additions have precedence over concurrent deletions) is selected for both source reactives. Appointments from both calendars are tracked in the `all_appointments` derived reactive

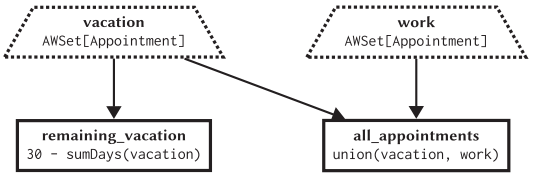


Fig. 1. Data-flow graph of the calendar application.

(Line 5), while the `remaining_vacation` reactive (Line 6) tracks the number of remaining vacation days. The *data-flow graph* of the application, where nodes are reactives and edges represent the derivation relation—in the direction of data flow—is visualized in Figure 1. This data-flow graph is created by the LoRe compiler and managed by its runtime.

2.2 Interactions

Changes to the state of the system, e.g., adding appointments to a calendar, happen through explicit *interactions*. Each interaction has two sets of type parameters: the types of source reactives that it modifies and the types of parameters that are provided when the interaction is applied. For example, the `add_appointment` interaction in Line 8 modifies a reactive of type `Calendar` and takes a parameter of type `Appointment`. The semantics of an interaction `I` are defined in four parts: (1) `requires` (Line 9) defines the preconditions that must hold for `I` to be executed, (2) `executes` (Line 11) defines the changes to source reactives, (3) `ensures` (Line 12) defines the postconditions that must hold at the end of `I`'s execution, (4) `modifies` (Line 13) defines the source reactives that `I` changes. The parameters of `requires`, `executes`, and `ensures` are functions that take the modified reactives and the interaction parameters as input (`cal` is of type `Calendar` and `a` is of type `Appointment`). The splitting of the definition of interactions in four parts allows for modularization and reuse. For instance, `add_appointment` is only a partial specification of an interaction, missing the `modifies` specification. Both `add_work` (Line 15) and `add_vacation` (Line 13) specify complete interactions by adding `modifies` to `add_appointment`; they are independent interactions that differ only in their modifies set.

Interactions encapsulate reactions to input from the outside world (e.g., the callback in Line 18 that is triggered by the UI and applies the arguments to `add_vacation`). Applying an interaction checks the preconditions, and—if they are fulfilled—computes and applies the changes to the source reactives, and propagates them to derived reactives—all in a "transactional" way in the sense that all changes to affected reactives become observable at-once ("atomically"). Only source reactives are replicated between devices, while derived reactives are computed by each device individually. LoRe gurantees that executing interactions does not invalidate neither postconditions nor invariants.

2.3 Invariants and Conflicts

LoRe expects the developer to use *invariants*, introduced with the keyword `invariant`, to specify application properties that should always hold. Invariants are first-order logic assertions given to a verifier based on the Viper verification infrastructure [39]. Invariants can help uncover programming bugs and reveal where the eventually-consistent replication based on CRDTs could lead to safety problems.

For illustration, consider the invariants for the calendar application in Lines 20 and 23. The invariant in Line 20 requires that all appointments must start before they end. Notice, how the invariant can be defined without knowing the amount of calendars and the actual structure of the data-flow graph by simply referring to the `all_appointments` reactive. This invariant represents a form of input validation, and is directly ensured by `add_appointment` interactions, because

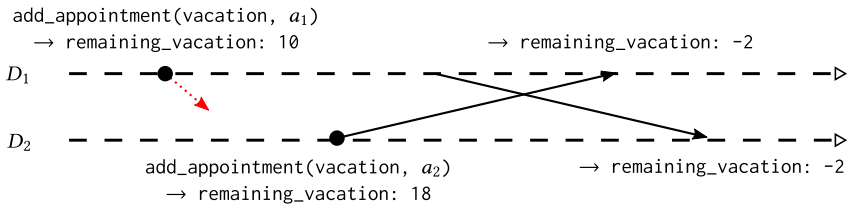


Fig. 2. Concurrent execution of interactions may cause invariant violations. In this example, device D_1 adds a vacation of 20 days to the calendar, while D_2 concurrently adds a vacation of 12 days. Given a total amount of 30 available vacation days, this leads to a negative amount of remaining vacation once the devices synchronize.

the precondition on the arguments requires the added appointment to start before it ends (Line 9). In absence of this precondition, the LoRe compiler would reject the program and report a safety error due to a possible invariant violation. The invariant in Line 23 requires that employees do not take more vacation days than available to them. Again, this is locally enforced by the precondition of the `add_vacation` interaction, which ensures that new entries do not exceed the remaining vacation days. But there is nothing stopping two devices from concurrently adding vacation entries, which in sum violates the invariant. Figure 2 illustrates such a situation: A user plans a vacation of 20 days on the mobile phone (device D_1) and later schedules a 12-day vacation on a desktop (device D_2), at a time when D_1 was offline. Thus, both interactions happened concurrently and after merging the states the calendar contains a total of 32 days of vacation, violating the `remaining_vacation` invariant.

This example illustrates a conflict between concurrent interactions that can potentially violate an invariant. In this case, two executions of `add_vacation` must not happen concurrently. A simple remedy for preventing conflicts and the resulting invariant violations is coordinating (synchronizing) the involved interactions. In the given example, this would mean that two devices could never add vacation entries concurrently, because every execution of `add_vacation` would need to be coordinated with the other devices.

LoRe's verifying compiler reports conflicting interactions to the developer and automatically generates the required coordination code for the execution of such interactions (see Section 4.3). The generated coordination code employs a token-based approach [18], which ensures that devices can only execute conflicting interactions iff they hold the tokens for the interaction itself *and* all its conflicts (in this case, they would only need the `add_vacation` token). Every token can only ever be held by one device at a time, effectively preventing every other device from executing conflicting interactions until they are able to coordinate with the token-holding device and can obtain the required token(s).

This approach ensures program safety but can be at odds with one of the principles behind local-first software: offline-availability. Usually, adding stronger guarantees will lead to more synchronization, ultimately resulting in an availability/safety trade-off. In practice, one can improve the situation by employing more sophisticated token distribution algorithms and timeouts[3] but this still blocks certain devices from performing certain actions at a given time. LoRe's conflict reporting helps developers to explore these trade-offs and allows them to make informed decisions about the safety guarantees of their program. When they find that their program requires too much synchronization, they can lower the guarantees by adapting their invariants.

[3]One example would be assigning a primary device that manages the tokens. This could mean, e.g., that the user's laptop is always able to perform all interactions (even when offline) while the user's phone needs to be able to contact the laptop for certain conflicting interactions.

$$
\begin{array}{rll}
P & ::= (A, \delta, I, (D \mid \ldots \mid D)) & \textit{Program} \\
D & ::= \langle \sigma, L \rangle & \textit{Device} \\
A & ::= \{\text{Interaction}((r, \ldots, r), l, l, t)\} & \textit{Interactions} \\
\sigma & ::= \{\text{val } r = \text{Source}(v)\} & \textit{Sources} \\
\delta & ::= \{\text{val } r = \text{Derived}(t)\} & \textit{Derived} \\
I & ::= \{\text{Invariant}(l)\} & \textit{Invariants} \\
v & ::= r \mid \text{true} \mid \text{false} \mid x \Rightarrow t & \textit{Value} \\
t & ::= v \mid t\ t \mid x \mid & \lambda\ \textit{Term} \\
 & \quad r.\text{value} & \textit{Reactive Access} \\
l & ::= t \mid x \Rightarrow l \mid \neg l \mid l == l \mid l \wedge l \mid l \vee l \mid & \textit{Logic Term} \\
 & \quad \text{forall } x.\ l \mid \text{exists } x.\ l & \textit{Quantifier}
\end{array}
$$

$$
\begin{array}{rl}
x & \textit{Variables} \\
r & \textit{Reactive Identifiers} \\
L \subseteq A & \textit{Locks}
\end{array}
$$

Fig. 3. Abstract syntax of LoRe programs.

3 PROGRAMMING MODEL

This section formally presents the syntax and semantics of the programming model and discusses how we verify that execution of a program preserves safety guarantees specified by invariants. The definition of the program execution is split into a big-step semantics for handling reactive updates on a single device and a labelled transition system to model execution of the overall distributed system. LoRe guarantees that given a valid program state (that ensures the safety invariants), any step taken in the labelled transition system preserves the validity of the program. To preserve safety without sequentializing the whole distributed program execution, the formal semantics relies on an oracle that tells us when two interactions have a conflict. We then give a proof of safety preservation given our oracle. Using the insights of this proof, Section 4 shows how we use the verification infrastructure to compute this oracle.

3.1 Syntax and Evaluation Semantics

3.1.1 Syntax. Figure 3 shows the abstract syntax for LoRe. A distributed program P is defined as a tuple, whose elements are a set of interactions A, a set of derived reactives δ, a set of invariants I, and a list of devices $(D_1 \mid \cdots \mid D_n)$, using $\mid$ as a separator in reference to parallel composition. We write $D \in P$ to state that D is one of the devices in P, where a device $D = \langle \sigma, L \rangle$ consists of an assignment of source reactives σ and a set of locks L. For the following definitions, we use curly braces as part of the meta syntax to denote that an expression occurs zero or more times in any order. This is used for top-level definitions in LoRe, and we treat such expressions as having set semantics.

Every $\text{Interaction}((r_1, \ldots, r_n), l_{pre}, l_{post}, t_{exec})$ is composed of a list of affected reactives $(r_1, \ldots, r_n)$, pre- and postconditions l_{pre} and l_{post}, and the interaction body t_{exec}. We introduce an inner term language t for the bodies of reactives and interactions, which is a simple lambda-calculus extended with access to reactives. We write $v = \sigma(r)$ to refer to the current value of a source reactive, if the expression $\text{val } r = \text{Source}(v)$ is present in σ, and $t = \delta(r)$ to refer to the

$$\textsc{ValueSource}\ \frac{t \Downarrow_\sigma r \qquad v = \sigma(r)}{t.\text{value} \Downarrow_\sigma v} \qquad \textsc{ValueDerived}\ \frac{t \Downarrow_\sigma r \qquad t_d = \delta(r) \qquad t_d \Downarrow_\sigma v}{t.\text{value} \Downarrow_\sigma v}$$

Fig. 4. Semantics for reactive term evaluation.

$$\textsc{Interact}\ \frac{\begin{array}{c} a = \text{Interaction}((r_1, \ldots, r_n), l_{pre}, l_{post}, t_{exec}) \qquad a \in A \\ \mathit{conflicts}(a) \subseteq L \qquad (l_{pre}\ v_{arg}) \Downarrow_\sigma \text{true} \qquad (t_{exec}\ v_{arg}) \Downarrow_\sigma (v_1, \ldots, v_n) \\ (l_{post}\ v_{arg}) \Downarrow_{\sigma'} \text{true} \qquad \sigma' = \mathit{update}(\sigma, (r_1, \ldots, r_n), (v_1, \ldots, v_n)) \end{array}}{(D_1 \mid \ldots \mid \langle \sigma, L \rangle \mid \ldots \mid D_n) \xRightarrow[a]{v_{arg}} (D_1 \mid \ldots \mid \langle \sigma', L \rangle \mid \ldots \mid D_n)}$$

$$\textsc{Sync}\ \frac{\begin{array}{c} L \subseteq L_s \qquad L_s' = L_s \setminus L \qquad L_r' = L_r \cup L \\ \sigma_r' = \mathit{merge}(\sigma_r, \sigma_s) \end{array}}{\begin{array}{c} (D_1 \mid \ldots \mid \langle \sigma_s, L_s \rangle \mid \ldots \mid \langle \sigma_r, L_r \rangle \mid \ldots \mid D_n) \xRightarrow[sync]{L} \\ (D_1 \mid \ldots \mid \langle \sigma_s, L_s' \rangle \mid \ldots \mid \langle \sigma_r', L_r' \rangle \mid \ldots \mid D_n) \end{array}}$$

$$\mathit{update}(\sigma, (r_1, \ldots, r_n), (v_1, \ldots, v_n))(r) = \begin{cases} \mathit{merge}_r(\sigma(r), v_i) & \text{if } r = r_i \\ \sigma(r) & \text{otherwise} \end{cases}$$

$$\mathit{merge}(\sigma_1, \sigma_2)(r) = \mathit{merge}_r(\sigma_1(r), \sigma_2(r))$$

$$\mathit{conflicts} : A \to \mathbb{P}(A)$$

Fig. 5. Semantics for interactions and device communication.

body of a derived reactive if the expression val r = Derived(t) is present in δ. We use a first-order logic language l, which embeds λ-terms, for defining invariants, pre-, and postconditions.

A, δ, and I are static and only devices $D = \langle \sigma, L \rangle$ may change during execution by updating the values of source reactives σ or their currently held locks L. Semantically, each interaction $a \in A$ has a one-to-one correspondence to a lock, thus we syntactically represent locks the same as interactions with $L \subseteq A$.

3.1.2 Term Evaluation. We use a big-step semantics for term evaluation. Figure 4 shows the rules for reactive evaluation—we omit rules related to standard lambda-calculus and logic evaluation. We write $t \Downarrow_\sigma v$ to state that t evaluates to v given a current assignment of source reactives σ. Note that evaluation depends on δ, but we omit writing it in the rules as it is fixed for each program and thus always clear from context. Rule `ValueSource` retrieves the current value of a source reactive from the store σ. Rule `ValueDerived` evaluates the expression of a derived reactive, which may depend on other reactives, thus the rule potentially results in evaluating many derived and source reactives in the data-flow graph.

3.1.3 Semantics for Interactions and Device Communication. Figure 5 presents the semantics for interactions and device communication together with auxiliary functions. We use three auxiliary functions *update*, *merge* and *conflicts*. Function *update* takes a set of source reactives σ, a tuple of reactive identifiers, a tuple of values and returns a new set with updated values. The value of each source reactive is a CRDT—we use a state-based model in our formalization: When a source reactive is updated, the new value is computed by merging the update into the current value. The function merge_r is defined by the CRDT stored in r.

Function *merge* takes two stores σ_1 and σ_2 and merges them by pair-wise merging the values of each source reactive through $merge_r$. The *update* and *merge* functions on stores are commutative, associative and idempotent, because $merge_r$ also has these properties by the definition of CRDTs. Notably, this implies a partial order on states where $\sigma \leq update(\sigma, (r_1, \ldots, r_n), (v_1, \ldots, v_n))$ and $\sigma_1 \leq \sigma_2$ if $merge(\sigma_1, \sigma_2) = \sigma_2$. We will use this order to reason about cases where σ_2 includes all changes of σ_1. For syntactic convenience, we lift the order to devices $D_{1/2} = \langle \sigma_{1/2}, L_{1/2} \rangle$, where we write $D_1 \leq D_2$ if $\sigma_1 \leq \sigma_2$.

The *conflicts* function takes an interaction and returns the set of interactions that conflict with it. If an interaction a_1 has no conflicts, then $conflicts(a_1) = \emptyset$; if there is an interaction a_2 that conflicts with a_1, then $\{a_1, a_2\} \subseteq conflicts(a_1)$ and $\{a_1, a_2\} \subseteq conflicts(a_2)$. This ensures that whenever a device wants to execute an interaction with conflicts, it has to hold the locks for the interaction itself and for all the conflicting interactions. The *conflicts* function serves as an oracle to prevent concurrent execution of interactions. Semantically, it defines the synchronization requirements of the program. Specifying an empty conflict set for every interaction, induces no synchronization, thus providing only causal consistency, while specifying an interaction as conflicting with every other interaction yields sequential consistency, disallowing any concurrent executions. We show in Section 4 how to compute a conflict function that prevents only those concurrent executions that would result in invariant violations on merged state.

We use a labelled transition system to model program execution, where transitions are labelled by interactions or synchronizations, which both occur non-deterministically. The `Interact` rule defines the semantics for applying an interaction a with an argument v_{arg}. The definition assumes that l_{post}, l_{pre}, and t_{exec} are functions; otherwise a is ill-defined and cannot be executed. The execution of a moves a device from state $\langle \sigma, L \rangle$ to state $\langle \sigma', L \rangle$, if: (1) the precondition l_{pre} applied to v_{arg} evaluates to true before the execution, (2) the device executing a holds the locks of all $conflicts(a)$, (3) the postcondition l_{post} applied to v_{arg} evaluates to true after the execution, (4) and evaluating t_{exec} returns a tuple[4] of new values $(v_1, \ldots, v_n)$ for each source reactive $(r_1, \ldots, r_n)$, which are used to update the store of source reactives σ.

The rule `Sync` models communication between devices. Synchronization happens between a sending device $D_s = \langle \sigma_s, L_s \rangle$ and a receiving device $D_r = \langle \sigma_r, L_r \rangle$, where the former transfers state and locks to the latter. A set of locks $L \subseteq L_s$ is removed from D_s and added to the locks of D_r. The state σ_r is merged with the sent state σ_s. By combining state updates and lock exchange in a single transition, we ensure that for every interaction a that needs coordination, a device D_r, which receives the locks for a, also receives the effects of the last application of a. Moreover, merging the state of all source reactives in a single step ensures causal consistency for the entire state of the application, not only for single reactives (i.e., single CRDTs).

An interaction only changes a single device, hence, we can abbreviate applications of `Interact` to $D \xRightarrow[a]{v_{arg}} D'$ to express that a device D executed an interaction regardless of the state of the other devices. Similarly, we write $D_s \xRightarrow[sync]{D_r} D'_r$ to express that D_s was synchronized into D_r producing D'_r (and D'_s, which we do not need to state explicitly, as it is fully defined by the other three devices).

3.2 Verifying Program Safety

3.2.1 Definitions. LoRe guarantees that program execution is safe: A program in a state where all safety invariants hold, transitions only into states where the invariants still hold. A given program state that satisfies all safety invariants is called valid. Formally:

[4]Using a suitable encoding of tuples in the lambda calculus.

Definition 3.1 (Validity). Given P with invariants I and devices $D_1, \ldots, D_n$, we say that P is valid, written $valid(P)$, if $valid(D_i), i \in \{1, \ldots, n\}$. A device $D = \langle \sigma, L \rangle$ is *valid*—written $valid(D)$—if for any Invariant$(l) \in I$, $l \Downarrow_\sigma$ true.

Definition 3.2 (Safety). A program P is *safe*, if for any possible transition $P \Rightarrow P'$, $valid(P) \Rightarrow valid(P')$.

To enable automatic and modular verification, we break safety checking into two simpler properties on invariants that can be automatically checked by Viper: *invariant preservation* and *confluence*. Invariant preservation ensures the safety of individual interactions, whereas confluence (adapted from Bailis et al. [5] and other works that build on the CALM theorem [2, 19]) relates two interaction executions (of the same or different interactions). We show how to mechanically prove these properties using Viper in Section 4. The rest of this section formally introduces all mentioned properties and proves soundness of our approach by showing that safety preservation follows from invariant preservation of all interactions, and confluence of all interactions that are not marked as conflicting.

Definition 3.3 (Invariant Preservation). An interaction a is *invariant preserving*, written $preserving(a)$, if $(valid(D) \wedge D \overset{v_{arg}}{\underset{a}{\Longrightarrow}} D') \Rightarrow valid(D')$, i.e., given a valid device D and some argument v_{arg}, the execution of a in D produces a valid device D'.

Definition 3.4 (Confluence). Interactions a_1 and a_2 are *confluent*, written $confluent(a_1, a_2)$, if for any valid devices $D_i = \langle \sigma_i, L_i \rangle$ and $D_j = \langle \sigma_j, L_j \rangle$, and any argument values v_1 and v_2:

$$
\begin{aligned}
& D_i \overset{v_1}{\underset{a_1}{\Longrightarrow}} \langle \sigma_i', L_i \rangle \wedge D_j \overset{v_2}{\underset{a_2}{\Longrightarrow}} \langle \sigma_j', L_j \rangle \wedge \\
& D_{m_1} = \langle merge(\sigma_i, \sigma_j'), L_i \cup L_j \rangle \wedge \\
& D_{m_2} = \langle merge(\sigma_i', \sigma_j), L_i \cup L_j \rangle \wedge \\
& merge(\sigma_i', \sigma_j') = \sigma' \ \wedge \ L_i \cap L_j = \emptyset \\
\Rightarrow & D_{m_1} \overset{v_1}{\underset{a_1}{\Longrightarrow}} \langle \sigma', L_i \cup L_j \rangle \wedge \\
& D_{m_2} \overset{v_2}{\underset{a_2}{\Longrightarrow}} \langle \sigma', L_i \cup L_j \rangle.
\end{aligned}
$$

Confluence states that applying interactions a_1, a_2 on two devices leads to the same results, no matter if a synchronization happens in-between or after the interactions. Thus, concurrent execution of the two interactions leads to the same result as sequential execution. Note that a_1 and a_2 are always trivially confluent if they affect disjoint subsets of source reactives and a_1 can never invalidate a_2's precondition and vice versa. In this case, sequential execution of the two interactions (in any order) always has the same result as concurrent execution. We leverage this insight to reduce the amount of confluence proofs generated by our implementation in Section 4.1.

3.2.2 Soundness. Given the definitions of invariant preservation and confluence, we now move on to prove soundness of LoRe's execution model.

Definition 3.5 (Initial Program). The initial program P^0 has devices $(D_1^0 \mid \cdots \mid D_n^0)$ that all have the same initial state of source reactives and all locks are with device D_1^0, i.e., $D_1^0 = \langle \sigma, A \rangle \wedge D_i^0 = \langle \sigma, \emptyset \rangle, \forall i \in \{2, \ldots, n\}$.

Lemma 3.6 (Correct Locking). *The locking mechanism ensures for any program execution $P^0 \Longrightarrow \cdots \Longrightarrow P^m$ that conflicting interactions are sequentially ordered. Specifically, for any two*

conflicting interactions a_1, a_2 with transitions $\langle \sigma_1, L_1 \rangle \overset{v_1}{\underset{a_1}{\Rightarrow}} \langle \sigma_1', L_1 \rangle$ and $\langle \sigma_2, L_2 \rangle \overset{v_2}{\underset{a_2}{\Rightarrow}} \langle \sigma_2', L_2 \rangle$, and $\mathit{conflicts}(a_1) \cap \mathit{conflicts}(a_2) \neq \emptyset$, the starting state of either a_1 or a_2 must include all changes produced by the other: $\sigma_2' \leq \sigma_1$ or $\sigma_1' \leq \sigma_2$.

Proof. We first show by induction that any program state that is reachable from the initial program P_0 assigns each lock to exactly one device. This is true for the initial program, interact transitions do not modify the lock assignment, and the sync rule removes the same set of locks from the sending devices that are added to the receiving device.

Then, we prove that non-overlapping locks ensure Lemma 3.6. Conflicting interactions require the same lock $l \in \mathit{conflicts}(a_1) \cap \mathit{conflicts}(a_2)$. This lock must be present on both devices executing the conflicting interactions, i.e., $l \in L_1$ and $l \in L_2$. Thus, the lock must be transferred from $D_1' = \langle \sigma_1', L_1 \rangle$ to $D_2 = \langle \sigma_2, L_2 \rangle$ (or symmetrically from D_2' to D_1), using any number of transitions. By commutativity, associativity and idempotence of the *merge* function, each sync transition ensures that the receiving device D_r has a state σ_r' with $\sigma_s \leq \sigma_r'$ where σ_s denotes the state of the sending device. Similarly, every `interact` transition $\langle \sigma, L \rangle \overset{v}{\underset{a}{\Rightarrow}} \langle \sigma', L \rangle$ ensures that $\sigma \leq \sigma'$. This implies that either $\sigma_2' \leq \sigma_1$ or $\sigma_1' \leq \sigma_2$. □

Theorem 3.7 (Soundness). *Given a program P with interactions A and invariants I, if the verifier has shown that (i) $\forall a \in A.\ \mathit{preserving}(a)$, and (ii) $\forall a_1, a_2 \in A$, $\mathit{confluent}(a_1, a_2) \vee a_2 \in \mathit{conflicts}(a_1)$, then P is safe.*

Proof. To prove soundness, we show that for any sequence C of transitions $P^0 \Longrightarrow \cdots \Longrightarrow P^m$, $\mathit{valid}(P^0)$ implies $\mathit{valid}(P^m)$. To show $\mathit{valid}(P^m)$, we show that $\forall D_i^m \in P.\ \mathit{valid}(D_i^m)$. We show that an arbitrary $D^m \in P^m$ is valid, by showing that we can construct a serialization order $\mathcal{S}_i$ of interaction transitions $D_1^0 \overset{v_1}{\underset{a_1}{\Rightarrow}} \cdots \overset{v_l}{\underset{a_l}{\Rightarrow}} D'$ that starts from the initial device state with all locks, consists exclusively of applications of the `Interact` rule, and yields a device $D' = \langle \sigma_i^m, L \rangle$ with the same state of source reactives σ_i^m as D_i^m.[5] Given such $\mathcal{S}_i$ and $\forall a \in \mathcal{S}_i.\ \mathit{preserving}(a)$ (which follows from premise (i) of the Theorem), we can conclude that $\mathit{valid}(D')$. In turn, $\mathit{valid}(D')$ implies $\mathit{valid}(D_i^m)$, because validity only depends on σ_i^m. In other words, we show that every possible device state that is a result of interactions and synchronizations, could also be constructed through a sequence of only local interactions. Since every interaction on itself is *invariant preserving* (premise (i)), this implies that the program is safe.

Serial order construction. Let $D^m \in P^m$ be an arbitrary device that we choose to serialize. As a convention, we use D (without the superscript) to refer to that device, in any program state belonging to C or $\mathcal{S}$, e.g., an interaction on D would be a step of the form $D^k \overset{v}{\underset{a_1}{\Rightarrow}} D^{k+1}$. We construct the serial order $\mathcal{S}$ for D stepwise from the concurrent order C, by picking transitions at the end of C and prepend them to $\mathcal{S}$. We say that the last (rightmost) transition in C is the focused transition T. Below, we consider all possible rule applications that trigger T in a case-by-case way.

Case 1. T is an interaction $D \overset{v}{\underset{a}{\Rightarrow}} D'$. We prepend T to $\mathcal{S}$ and discard it from C.

Case 2. T is an interaction $D_i \overset{v}{\underset{a}{\Rightarrow}} D_i'$, $D_i \neq D$. We discard T from C, because by being the last transition in C, it does not affect D.

[5]Note that the length of C and $\mathcal{S}_i$ may differ, and $\mathcal{S}_i$ does not necessarily include all interactions of C.

Case 3. T is a synchronization $D_s \overset{D_r}{\underset{sync}{\Longrightarrow}} D'_r$ from any D_s (possibly D) to $D_r \neq D$. We discard T from C, because we know that D'_r will not synchronize with D after T.

Case 4. T is a synchronization $D_i \overset{D}{\underset{sync}{\Longrightarrow}} D'$ from any $D_i \neq D$ to D. To handle this case, we consider possible states of D_i and D before T occurred, especially whether devices had concurrent changes since their last synchronization:

Case 4.1. $D_i \leq D$, i.e., there are no changes to D_i compared to D. We discard T, because it only transfers some lock, and locks are irrelevant for the final serialization order.

Case 4.2. $D \leq D_i$, i.e., there are no changes to D compared to D_i. Similar to case 4.1, but now all relevant changes are on D_i. Because T is a sync, we know that the state of D after T is equal to D_i (except locks). We discard T and continue the construction with D_i as the chosen device.

Case 4.3. Both D and D_i have concurrent changes and the transition that produced the state of D preceding T is the application of an interaction a. We know that any interaction a is either confluent to any other interaction (including itself) or the other interaction is included in *conflicts*(a) (premise (ii) of the Theorem). Given Lemma 3.6, this implies that all interactions on D_i that are concurrent to a are confluent with a. Thus, we can use the confluence definition to reorder C to put the application of a at the rightmost end of C, directly after the synchronization T. After the reordering, we handle a according to case 1.

Case 4.4. Both D and D_i have concurrent changes and the state of D was produced by a synchronization T_k, $D_k \overset{D'}{\underset{sync}{\Longrightarrow}} D$, that synchronizes a third device $D_k \neq D_i$ into D. In other words, the effect of the last two transitions is a merge between D_k, D_i, and D, and we (potentially) have concurrent changes from all three for which we must find a single serialization order. On a high-level, we (arbitrarily) choose to order concurrent interactions on D first, D_i second, and D_k third by changing the transitions to first synchronize D_k into D_i and then D_i into D. In other words, we change T_k (the sync from D_k to D) in C to $T'_k = D_k \overset{D'}{\underset{sync}{\Longrightarrow}} D_i$.[6] This does not change the result state of D—it is still a result of merging the states of D, D_i, and D_k, where the change in merge order does not matter, because merging is associative and commutative.

Closing remarks. Our construction terminates, because (a) there is a finite amount of transitions in C and (b) each case above, except 4.4, reduces the size of C. Case 4.4 does not reduce the size of C, but it is impossible to indefinitely repeat it, as this would entail that there are indefinitely many synchronizations from D_k into D, which is impossible as C is assumed to be finite.

Also, the construction handles all situations, because cases 1 and 2 cover all possible applications of interactions, while cases 3 and 4 cover all applications of synchronizations—these are the only two rules that produce transitions. We know that the distinction in case 4 is complete, because cases 4.1 and 4.2 handle all situations where there are no concurrent changes[7] on one of the devices. If there are concurrent changes, then cases 4.3 and 4.4 again exhaustively cover that situation. In

[6] This disregards changes in transferred locks, but at this point those are irrelevant for the serialization order.

[7] Technically, both devices could apply confluent interactions that lead to the same result state—these are irrelevant due to idempotence of merges and thus can be discarded from the serialization.

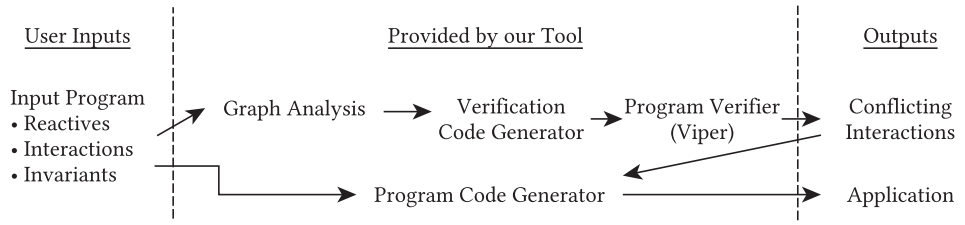


Fig. 6. Overview of LoRe's automated compilation and verification procedure.

particular, the one exclusion from case 4.4, specifically the situation that $D_k = D_i$ is covered by case 4.2, because then D_i is synchronized into D twice, without any changes to D in between.

In summary, by successively replacing and discarding transitions according to the cases above, we can generate a sequential order for any device $D_i^m \in P^m$, from which follows (using invariant preservation) that $valid(D_i^m)$. As we can construct such a serialization (which must not be the same) for all devices, we know that $valid(P^m)$. □

4 IMPLEMENTATION

Figure 6 depicts the architecture of LoRe's verifying compiler. The input to the compiler is a program with its specifications expressed by the invariants, e.g., the program in Listing 1. The output consists of the conflicting interactions and a safe executable program. To verify safety, we prove invariant preservation (Definition 3.3) for all interactions (a failed proof results in a compilation error) and try to prove invariant confluence (Definition 3.4) for all pairs of interactions. We employ an analysis of the data-flow graph to minimize confluence proof obligations to those invariant pairs that may actually conflict. Non-confluent interaction pairs are included in the conflict output. We use Viper for the verification step, but any other verification could be used and would still benefit from our minimization of the proof obligations. The implementation uses REScala, CRDTs, and a token-based locking protocol for generating a safe executable program. But they could be replaced by other implementations of data-flow programming, eventual consistency, and consensus with the same guarantees. The rest of this section describes the pipeline from Figure 6 in detail—from left to right, top to bottom.

4.1 Graph Analysis

Checking all pairs of interactions for confluence would result in an exponential amount of proof obligations. To avoid this, we employ a graph analysis to quickly detect pairs of interactions that cannot conflict, because they change completely separate parts of the data-flow graph. The graph analysis algorithm (Figure 7) checks every interaction to determine its reachable reactives: The source reactives that are affected by the interaction together with all transitively derived reactives that depend on them (Line 12). Next, the algorithm determines the *overlaps* between interactions and invariants (Line 4). An interaction and an invariant *overlap* if any reactive occurring in the invariant is part of the interaction's reachable reactives. Two interactions *overlap* if there is at least one invariant they both overlap with. Only overlapping interactions produce proof obligations.

For illustration, consider the `add_work` interaction. It modifies the `work` reactive, and—transitively—`all_appointments`. Hence, the reachable reactives are {*work*, *all_appointments*} and only the first but not the second invariant in Listing 1 overlaps. Thus, neither the `remaining_vacation` reactive, nor the invariant on this reactive will be part of the proof obligation for the `add_work` interaction.

```
1  Procedure overlapAnalysis:
2      for each interaction do
3          determine overlappingInvariants(interaction)
4  Function overlappingInvariants(interaction):
5      subgraph ← reaches(interaction)
6      ℐ ← ∅
7      for each invariant do
8          ℛ ← all reactives that this invariant mentions
9          if ℛ ∩ subgraph ≠ ∅ then
10             ℐ ← ℐ ∪ { invariant }
11     return ℐ
12 Function reaches(interaction):
13     ℛ ← all reactives that this interaction modifies
14     subgraph ← ℛ
15     for each r ∈ ℛ do
16         children ← all reactives that transitively depend on r
17         subgraph ← subgraph ∪ children
18     return subgraph
```

Fig. 7. Pseudocode of the algorithm used to determine which invariants overlap with a transaction.

```
1  // GRAPH
2  // sources
3  field work: AWSet[Appointment]
4  field vacation: AWSet[Appointment]
5  // derived
6  define all_appointments(work, vacation) toSet(work) union toSet(vacation)
7  define remaining_vacation(vacation) 30 - countDays(toSet(vacation))
8
9  // INVARIANTS
10 define inv_1(all_appointments) forall a: Appointment :: a in all_appointments
          ==> get_start(a) < get_end(a)
11 define inv_2(remaining_vacation) remaining_vacation >= 0
```

Listing 2. Viper representation of the graph and invariants of the calendar example.

4.2 Automated Verification

4.2.1 Translation to Viper's Intermediate Language. Listing 2 illustrates how we represent the data-flow graph and the safety invariants of our calendar example (Listing 1) in Viper's intermediate verification language [39]. We represent the data-flow graph as a mutable object with one field per source reactive (Line 3). In Viper, objects are implicitly defined by which fields the program accesses. Derived reactives (Line 6) and invariants (Line 10) are expressed as Viper macros—pure functions that describe the invariant or body of the reactive, and receive the reactives they depend on as function inputs.

Given these definitions, we synthesize one Viper *method* per interaction, based on the formal evaluation semantics in Section 3. Viper verifies programs on a per-method basis where methods represent a sequential computation annotated with pre- and postconditions. Listing 3 shows the Viper method for the `add_work` interaction. Pre- and postconditions of interactions are simply translated to pre- and postconditions of the Viper methods (Lines 7 and 12) while the `executes` part of each interaction is represented by the method body (Line 19). Reading derived reactives is

```
1  method add_work(
2    graph: Ref,
3    appointment: Appointment)
4  // permissions
5  requires acc(graph.work, 1) && requires acc(graph.vacation, 1/2)
6  // preconditions
7  requires !(appointment in toSet(graph.work))
8  requires get_start(appointment) < get_end(appointment)
9  // overlapping invariants
10 requires inv_1(all_appointments(graph.vacation, graph.work))
11 // postconditions
12 ensures acc(graph.work, 1) && acc(graph.vacation, 1/2)
13 ensures appointment in toSet(graph.work)
14 ensures size(graph.work) == old(size(graph.work)) + 1
15 ensures toSet(graph.work) == old(toSet(graph.work)) union Set(appointment)
16 // overlapping invariants
17 ensures inv_1(all_appointments(graph.vacation, graph.work))
18 {
19   graph.work := add(graph.work, appointment)
20 }
```

Listing 3. Viper representation of the add_work interaction.

modeled by inlining the respective Viper macro, which results in our big-step evaluation semantics. Additionally, we include every overlapping invariant as pre- and postconditions (Lines 10 and 17) so that the verifier can prove invariant preservation. Evaluating invariants is expressed by nested application of the previously defined macros (Listing 2) to the source reactives, corresponding to the propagation of values though the data-flow graph. Viper uses explicit permissions for shared state, thus we explicitly pass a reference to the data-flow graph (Line 2); it declares write permissions for all reactives that are modified by the interaction and read permissions for the reactives that are only accessed as part of the invariants (Line 5).[8]

4.2.2 Proving Invariant Preservation and Confluence. Given the Viper encoding of each interaction—which includes overlapping invariants—Viper directly outputs verification results that correspond to invariant preservation (Definition 3.3). Any verification errors at this stage are errors in the supplied specification or programming bugs and must be addressed by the programmer.

To check for confluence the compiler creates a Viper method for each overlapping pair of interactions. Such a method models the specification for invariant confluence (Definition 3.4). Any verification errors here indicate that the two interactions are non-confluent. As our soundness proof requires either confluence of interactions, or their inclusion in the set of conflicting interactions, safety is ensured by marking all non-confluent pairs of interactions as conflicting. Developers should still check the conflicts to ensure that they are not due to a bug or specification error.

4.3 Synchronization at Runtime

Our compiler generates an executable application by converting the data-flow graph to a distributed REScala program [37, 38]. REScala supports all reactive features we require and integrates well with our CRDT-based replication, but it has no mechanism for synchronization. LoRe's

[8]Viper uses *fractional permissions* where an `acc(n)` statement with any $n < 1$ corresponds to a read-permission and a statement with $n = 1$ corresponds to a write-permission.

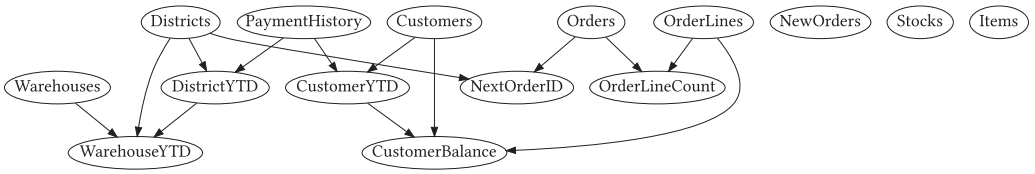


Fig. 8. Data-flow graph of the TPC-C benchmark application.

formal synchronization semantics (cf. Section 3) could be implemented using any existing form of coordination, such as a central server, a distributed ledger, a consensus algorithm, or a distributed locking protocol. Which choice is suitable, depends on the target application, network size, and the reliability of the chosen transport layer. We use a simple distributed locking protocol for our prototype implementation: Each interaction has an associated lock (represented as a simple token). Whenever a device wants to execute an interaction, it acquires the tokens of all conflicting interactions. If multiple devices request the same token concurrently, then the token is given to the device with the lowest ID that requested it. This ensures deadlock freedom; fairness is left for future work. After performing the interaction, the resulting state changes are synchronized with the other devices and the tokens are made available again. Timeouts ensure that whenever a device crashes or becomes unavailable for a longer period of time, its currently owned tokens are released and any unfinished interactions by the device are aborted.

5 EVALUATION

Our evaluation aims to validate two claims about LoRe's programming model:

C1: It facilitates the development of safe local-first software.
C2: It enables an efficient and modular verification of safety properties.

We base our validation on two case studies. First, we implemented the standard TPC-C benchmark [49] as a local-first application in LoRe. This case study enables comparing LoRe's model with traditional database-centered development of data processing software and showcasing the benefits of LoRe's verifiable safety guarantees on standard *consistency conditions*. Second, we implemented the running calendar example (Section 2) using Yjs [42]. This case study allows comparing LoRe with an existing framework for local-first applications that we consider a representative of the state-of-the-art.

5.1 Does LoRe facilitate the development of safe local-first software?

5.1.1 Local-first TPC-C. TPC-C models an order fulfillment system with multiple warehouses in different districts, consisting of five *database transactions* alongside twelve *consistency conditions*. We implemented TPC-C in LoRe by mapping database tables to source reactives and derived database values to derived reactives. Each database transaction was modeled as a LoRe interaction.

In the LoRe implementation of TPC-C, each warehouse holds a local copy of the data on which transactions—modeled as interactions—are executed before being synchronized with other warehouses.

Reactives. Figure 8 shows the reactive graph of the application. The structure roughly follows the database structure described in TPC-C. We represent each database table by a source reactive and model derived database values as derived reactives. For example, the `DistrictYTD` reactive in Listing 4 represents the **year-to-date (YTD)** balance of districts. After reading the `districts` reactive (Line 2), we perform the following steps for each district: We read the relevant entries in

```
1 val districtYTD: Derived[Map[District, YTD]] = Derived {
2   districts.map{d =>
3     val payments = paymentHistory.filter(ph => sameDistrict(ph, d))
4     val ytd = payments.sum()
5     (d, ytd)
6   }.toMap() }
```

Listing 4. LoRe representation of the derived `DistrictYTD` reactive (simplified).

```
1  val payment: Unit = Interaction[PaymentHistory][PaymentArgs]
2  .modifies(paymentHistory)
3  .executes{ph => pa => {
4     val c: Customer = findCustomer(customers, pa.c_id, pa.c_last)
5     val id: Id = getId(c)
6     ph.add(new_payment(id, pa))}
7  }
8  .requires{ph => pa => pa.h_amount > 0 && exists c: Customer ::
9     (c in customers.toSet && (c.id == pa.c_id || c.last == pa.c_last))}
10 .ensures{ph => pa => exists p: Payment ::
11      toSet(ph) == old(toSet(ph)) union Set(p)}
12 .ensures{ph => pa => old(ph.toSet).subset(ph.toSet)}
```

Listing 5. LoRe representation of TPC-C's payment interaction (simplified).

the payment history (Line 3), calculate the sum of the YTD values (Line 4), and return the result as a single entry mapping from district to YTD (Line 5).

Interactions. We implement TPC-C transactions as interactions. For illustration, consider the `payment` interaction in Listing 5, which is applied whenever a customer pays a certain amount to the system. Payments are not associated to a specific order and are simply stored in the payment history. Table modifications in TPC-C have multiple arguments, which we encapsulate into an argument of type `PaymentArgs` (Line 1). Applying the interaction (Line 3) retrieves the customer object matching the payment arguments by executing a function that accesses the `customers` reactive (Line 4). Line 6 updates and returns the new history. The preconditions (Line 8) encodes assumptions about the arguments, notably that a customer for whom we add the payment actually exists, and the postcondition (Line 12) describes the effect of adding a new payment.

Invariants. TPC-C defines 12 consistency conditions, of which 9 are consistency constraints between tables and derived tables. Their correctness is automatically ensured by LoRe without further specification. For example, the consistency condition 9 [49]:

> Entries in the DISTRICT and HISTORY tables must satisfy the relationship:
> D_YTD = sum (H_AMOUNT) for each district defined by
> (D_W_ID, D_ID) = (H_W_ID, H_D_ID).

directly corresponds to the definition of the `DistrictYTD` reactive in Listing 4 and is thus always true by design. Only the remaining three conditions require translating the natural language specification into first-order logic formulae to be used as invariants. As an example, consider consistency condition 5 from the TPC-C specification [49]:

> For any row in the ORDER table, O_CARRIER_ID is set to a null value if and only if there is a corresponding row in the NEW-ORDER table [...].

```
1  invariant forall o: Order :: o in Orders ==> (
2      o.carrier_id == 0 <==>
3      (exist no: NewOrder :: no in NewOrders && no.id = o.id) )
```

Listing 6. TPC-C's consistency condition 5 expressed as a LoRe invariant.

This condition cannot be represented as a derived reactive, because the carrier ID is not derived from other values but set explicitly whenever an order is shipped. Listing 6 displays the encoding in LoRe—an (almost) literal translation of the specification.

Comparison. The implementation effort for the core functionality of TPC-C in LoRe is comparable to a traditional design relying on a relational database model. While modeling the application using reactives might require some adaption from developers not familiar with data-flow programming, we found that using derived reactives led to a more concise and less error-prone design when compared to storing derived values in separate tables. This observation is supported by the fact that we only need to explicitly address 3 of 12 consistency conditions of TPC-C. We were able to phrase the remaining 3 conditions as invariants by directly translating the natural language formulations into logical specifications. To prove them, we additionally needed to specify pre- and postconditions of interactions corresponding to transactions (see Figure 5). Other than that, LoRe relieves the TPC-C developer from any considerations of transaction interleavings that could potentially violate the conditions as well as from implementing the synchronization logic, both tedious and error-prone processes. Moreover, unlike TPC-C, which cares only about the consistency of the database, LoRe treats consistency constraints uniformly from (shared) state to UI. This is enabled by the reactive programming paradigm, which also guarantees 9 of 12 consistency conditions by design.

5.1.2 Yjs-based Calendar. We now compare the LoRe implementation of the distributed calendar to an implementation using the state of the art local-first framework Yjs [42]. Like other solutions for local-first software, Yjs uses a library of CRDTs (usually maps, sets, sequences/arrays, and counters) composed into nested trees—called a *document*—used to model domain objects.

Source and Derived Variables. For illustration, consider Listing 7, showing how one could implement the domain model of the calendar application. Lines 2 and 3 initialize two CRDTs for the work and vacation calendar. Yjs has no abstraction for derived values and only provides callbacks for reacting to value changes, e.g., Lines 7–15 declare callback methods that update the derived variables in case the Yjs document changes.

Safety Guarantees. Using callbacks to model and manage complex state that changes both in time and in space has issues. It requires that developers programmatically update the derived values once the sources get updated, via local interactions or on receiving updates from other devices, with no guarantees that they do so consistently. It yields a complex control-flow and requires intricate knowledge of the execution semantics to ensure atomicity of updates, let alone to enforce application-level safety properties. Frameworks like Yjs do not offer support for application invariants and thus force developers to integrate custom safety measures at each possible source of safety violations. As an example, consider Listing 8 showing how the `addVacation` interaction could be implemented in Yjs. Lines 2 and 11 check the preconditions of the interaction, but as discussed in Section 2.3, such local checks are insufficient to maintain safety. In general, global reasoning without tool support quickly becomes infeasible, and even more so for designs based on callbacks. Once updates inducing invariant violations are propagated, it is difficult to "undo" them. Thus, programmers are required to either provide and coordinate compensation actions or to integrate mechanisms for strong synchronization on top of CRDT-based eventual consistency. Both approaches are difficult to get right, putting safety and/or availability at risk.

```
const ydoc = new Y.Doc()
let work = ydoc.getMap('work');
let vacation = ydoc.getMap('vacation'
        );
let all_appointments;
let remaining_vacation = 30;

work.observe(ymapEvent => {
 all_appointments = getMap(work,
        vacation);
})

vacation.observe(ymapEvent => {
 let days_total = getTotalVacDays(
        vacation);
 remaining_vacation = 30 - daysTotal;
 all_appointments = getMap(work,
        vacation);
})
```

Listing 7. Defining source and derived variables in Yjs.

```
function addAppointment(calendar,
        appointment) {
 if(appointment.start < appointment.
        end){
   calendar.set(appointment.id,
        appointment);
 }
}

function addVacation(appointment)
{
  let days =
    appointment.getDays();
  if(remainingVacation < days){
    console.log("Sorry, no vacation
        left!");
  }
  else{
    addAppointment(vacation,
        appointment)
  }
}
```

Listing 8. Adding appointments in Yjs.

In summary, while the replication capabilities of systems like Yjs are valuable for local-first applications, these systems still require the developer to do state management manually. The prevailing use of callbacks and implicit dependencies makes reasoning about the code challenging for both developers and automatic analyses. In contrast, LoRe allows declarative definitions of derived values, with positive effects on reasoning [14, 45]. Moreover, LoRe integrates application invariants as explicit language constructs, which allows for a modular specification and verification and relieves developers from having to consider every involved interaction whenever the specification changes.

5.1.3 On the Ergonomics of LoRe's Programming Model. While a detailed evaluation of LoRe's ergonomics is out of the scope of this article, in this subsection, we briefly outline why we expect LoRe's programming model to be easier to use than current models for local-first software. LoRe coherently combines the following main abstractions: *reactives*, *interactions*, and *invariants*, which provide programmers with a principled way to think about the data flow, (user) interactions, and safety guarantees of their program, respectively. None of these abstractions is in itself unfamiliar: First, CRDTs—the basis for LoRe's reactive data model—are popular data structures for modeling decentralized local-first software, available in libraries like Automerge [4], Collabs [51], or Yjs [42]. Second, reactive programming is a popular paradigm especially for interactive web applications and is present in widely used frameworks like React [21]. Past research has shown in user studies that **functional reactive programming (FRP)**, the variant of reactive programming used in LoRe can improve program comprehension when compared to a classic callback-driven programming style as it is present in vanilla JavaScript [14, 45]. LoRe combines CRDTs and FRP in one programming model. By doing so, LoRe makes it easier to reason about properties of compositions of CRDTs involved in an interaction. Finally, LoRe complements the CRDT-FRP combination with a *correctness-by-construction* approach [28] through its *invariants* and its verifying compiler.

By making invariants a first-class programming construct, LoRe encourages programmers to write programs alongside their specification. Given the specification, the verifying compiler helps programmers to refine their program until it meets their expectations. Depending on the

Table 1. Seconds to Verify Combinations of Interactions and Invariants of the two Example Applications

Distributed Calendar			
Interaction	**Invariant**		
	1	2	
Add vacation	3.32 (± 0.05)	2.97 (± 0.03)	
Remove vacation	3.28 (± 0.06)	3.00 (± 0.02)	
Change vacation	3.32 (± 0.05)	3.04 (± 0.03)	
Add work	3.31 (± 0.04)	—	
Remove work	3.30 (± 0.06)	—	
Change work	3.34 (± 0.06)	—	
TPC-C			
Interaction	**Consistency Condition**		
	3	5	7
New Order	45.4 (± 63.69)	7.63 (± 0.11)	14.49 (± 7.31)
Delivery	5.78 (± 0.03)	5.74 (± 0.07)	5.76 (± 0.09)

Each entry represents the mean verification time over five runs with the deviation shown in parentheses.

complexity of the application, this verification step introduces some additional specification burden when compared to approaches that do not rely on verification and thus offer weaker guarantees. In our experience, the specification burden of LoRe's model is modest and comparable to other deductive verification approaches [9, 30, 43] (see Section 5.1.1 for examples of the required specifications). It is also worth noting that in LoRe specifications and invariants are optional; they can thus be used depending on the desired strength of the correctness guarantees. This allows for an incremental workflow where specifications are added step-by-step to an existing codebase.

Summarizing our observations from Sections 5.1.1 and 5.1.2, LoRe's programming model proved very effective for our case studies and facilitated the development when compared to a classic relational data model (TPC-C) and existing local-first frameworks (Yjs). In our experience, the FRP model simplifies state management and composition of CRDTs, while the verification process helps port safety-critical applications like TPC-C to a decentralized setting.

5.2 Does LoRe Enable Efficient and Modular Verification of Safety Properties?

Safety invariants in LoRe are *global* in the sense that they must not be violated by *any* part of the program. But their enforcement is based on verifying individual local properties. We limit the need for verification to potential conflicts that we derive from the reactive data-flow graph. This optimization requires no further reasoning from the programmer and relies solely on the properties of the programming model. Programmers can add new functionality to the application (i.e., specify interactions) and only have to reason about the properties of that new functionality (i.e., specify its invariants) and the system ensures global safety—at only the cost of the amount of overlap with existing functionality. This allows a modular programming style where invariants and interactions are written by different developers and changes to the program are made incrementally.

To empirically evaluate the performance of LoRe's verifier, we quantify how long it takes to verify different combinations of interactions and invariants of our two case studies. The results are shown in Table 1. The calendar example has two additional types of interactions, which we have not shown in Section 2: removing and changing calendar entries. This leads to a total of six interactions (three per calendar reactive). As explained in the previous section, for TPC-C we only had to verify consistency conditions 3, 5, and 7. The benchmarks were performed on a desktop

PC with an AMD Ryzen 7 5,700G CPU and 32 GB RAM using Viper's *silicon* verification backend (release v.23.01) [50].

Results. Verification times differed depending mainly on the complexity and length of the transactions and invariants under consideration. Differences become apparent especially when looking at the results for TPC-C. Proofs involving the *New Order* interaction, which is the most "write-heavy" interaction of TPC-C that changes many source reactives at once, generally took longer to verify than others. For *New Order*, we also observe a much higher deviation of up to 64 s, which we assume to be caused by internal Z3 heuristics.[9] When interpreting the results, it is important to note that each interaction/invariant combination has to be verified only once and independently of other combinations. Large-scale applications can be verified step-by-step by splitting them into smaller pieces. Furthermore, we limit the need for verification to potential conflicts that we derive from the reactive data-flow graph. Programmers can add new functionality to the application (i.e., specify interactions) and only have to reason about the properties of that new functionality (i.e., specify its invariants) and the system ensures global safety—at only the cost of the amount of overlap with existing functionality. This allows for an incremental development style, where only certain parts of programs have to be (re-)verified, when they have been changed or added.

6 RELATED WORK

Our work relates to three areas: distributed data types, formal reasoning, and language-based approaches. Sections below relate work from each area to respective aspects of our approach.

6.1 Consistency Through Distributed Data Types

Conflict-free Replicated Data Types (CRDTs) [44, 47] are a building block for constructing systems and applications that guarantee eventual consistency. CRDTs are used in distributed database systems such as *Riak* [26] and *AntidoteDB* [1]. These databases make it possible to construct applications that behave under mixed consistency, but unlike our approach, they leave reasoning about application guarantees to the programmer. Gomes et al. [17] and Nair et al. [41] propose frameworks for formally verifying the correctness of CRDTs while *VeriFx* [11] and *Propel* [53] are dedicated language-based solutions for specifying and verifying replicated data types.

Several works suggest distributed data types that extend the capabilities of CRDTs: *Mergeable Replicated Data Types (MRDTs)* [23] automatically synthesize merge functions from relational specifications of the data type. *Katara* [29] synthesizes a verified CRDT from the specification of a sequential data type. De Porre et al. [13] suggest *strong eventually consistent replicated objects (SECROs)* relying on a replication protocol that tries to find a valid total order of all operations. Follow-up work on *explicitly consistent replicated objects (ECROs)* [12] reorders operations if possible and add coordination in cases where reordering is not sufficient. Similarly, *Hamsaz* [20] combines the specification of a sequential object with high-level invariants to synthesize a replicated object that satisfies the invariants. All approaches above tie consistency and safety properties to specific data types/objects. This is not sufficient to guarantee end-to-end correctness of an entire local-first application—consistency bugs can still manifest in derived information (e.g., in the user interface).

6.2 Automated Reasoning about Consistency Levels

Our formalization is in part inspired by the work of Balegas et al. [7 ,8] on *Indigo*. The work introduces a database middleware consisting of transactions and invariants to determine the ideal consistency level—called *explicit consistency*. They build on the notion of *invariant-confluence* for

[9]These could likely be improved by annotating quantifiers in invariants and pre-/postconditions with hand-crafted trigger expressions [39].

transactions that cannot harm an invariant, which was first introduced by Bailis et al. [5]. While they work on a database level, we show how to integrate this reasoning approach into a programming language (Section 3). An important difference between our *invariant-confluence* and the one by Balegas et al. [7] is that our approach also verifies local preservation of invariants, whereas their reasoning principle assumes invariants to always hold in a local context. In a more recent work called *IPA*, Balegas et al. [6] propose a static analysis technique that aims at automatically repairing transaction/invariant conflicts without adding synchronization between devices. We consider this latter work complementary to ours. Whittaker and Hellerstein [52] also build on the idea of invariant-confluence and extend it to the concept of *segmented invariant-confluence*. Under segmented invariant-confluence, programs are separated into segments that can operate without coordination and coordination only happens in between the segments. The idea is similar to our definition of *conflicting interactions*; however, their procedure cannot suggest a suitable program segmentation, but requires developers to supply them.

Gotsman et al. [18] propose an alternative formalization and proof rule, which allows reasoning about the invariant-safety of database transactions. Their formalization employs a token-based synchronization protocol similar to ours (see Section 3). The *SIEVE* framework [32] uses invariants and program annotations to infer where a Java program can safely operate under CRDT-based replication and where strong consistency is necessary. They do so by relying on a combination of static and dynamic analysis techniques. Compared to *SIEVE*, our formal reasoning does not require any form of dynamic analysis. *Blazes* [2] is another analysis framework that uses programmer supplied specifications to determine where synchronization is necessary to ensure eventual consistency. Unlike *Blazes*, LoRe ensures that programs are "by design" at least eventually consistent, while also allowing the expression and analysis of programs that need stronger consistency. *Q9* [22] is a bounded symbolic execution system, which identifies invariant violations caused by weak consistency guarantees. Similar to our work, *Q9* can determine where exactly stronger consistency guarantees are needed to maintain certain application invariants. However, its verification technique is bound by the number of possible concurrent operations. LoRe can provide guarantees for an unlimited amount of devices with an unlimited amount of concurrent operations (see Section 3).

6.3 Language Abstractions for Data Consistency

We categorize language-based approaches based on how they achieve consistency and on the level of programmer involvement.

Manual Choice of Consistency Levels. Approaches in this category expose consistency levels as language abstractions. Li et al. [33] propose *RedBlue Consistency* where programmers manually label their operations to be either blue (eventually consistent) or red (strongly consistent). In MixT [36], programmers annotate classes with different consistency levels and the system uses an information-flow type system to ensure that the requested guarantees are maintained. However, this still requires expert knowledge about each consistency level, and wrong choices can violate the intended program semantics. Other approaches [27, 40] expect programmers to choose between *consistency* and *availability*, again leaving the reasoning duty about consistency levels to the programmer. Compared to LoRe, languages in this category place higher burden on programmers: They decide which operation needs which consistency level, a non-trivial and error-prone selection.

Automatically Deriving Consistency from Application Invariants. Approaches in this category relieve programmers from reasoning about consistency levels based on some form of programmer annotations about application invariants. *CAROL* [31] uses CRDTs to replicate data and features a refinement typing discipline for expressing safety properties similar to our *invariants*. Carol makes

use of pre-defined data types with *consistency guards* used by the type system to check for invariant violations. The compatibility of datatype operations and consistency guards is verified ahead of time using an algorithm for the Z3 SMT solver. This approach hides much of the complexity from the programmer, but the abstraction breaks once functionality that is not covered by a predefined datatype is needed. Unlike Carol, LoRe does not rely on predefined consistency guards, but allows the expression of safety properties as arbitrary logical formulae. Additionally, *CAROL* only checks the concurrent interactions of a program for invariant violations, whereas LoRe verifies the overall application including non-distributed parts. Sivaramakrishnan et al. [48] propose *QUELEA*, a declarative language for programming on top of eventually consistent datastores. It features a contract-language to express application-level invariants and automatically generates coordination strategies in cases where invariants could be violated by concurrent operations. *QUELEA*'s contract-language requires programmers to express the desired properties using low-level visibility relations, which can be challenging to get right for non-experts. LoRe avoids this intermediate reasoning and automatically derives the right level of consistency for satisfying high-level safety invariants to enable end-to-end correctness.

Automating Consistency by Prescribing the Programming Model. Languages in this category seek to automate consistency decisions by prescribing a certain programming model such that certain consistency problems are impossible to occur. In *Lasp* [35], programmers model the data flow of their applications using combinator functions on CRDTs. Programs written in *Lasp* always provide eventual consistency but contrary to LoRe, *Lasp* does not allow arbitrary compositions of distributed data types. *Bloom* [3] provides programmers with ways to write programs that are *logically monotonic* and therefore offer automatic eventual consistency. Both *Lasp* and *Bloom*, however, are not meant to formulate programs that need stronger consistency guarantees. LoRe is similar to *Lasp* and *Bloom* in the sense that we also prescribe a specific—reactive—programming style. However, our programming model is less restrictive and allows arbitrary compositions of distributed datatypes. This is enabled by leveraging the composability properties of reactive dataflow graphs. Second, LoRe provides a principled way to express hybrid consistency applications with guarantees stronger than eventual consistency. Drechsler et al. [15] and Mogk et al. [37], ,38] also use a reactive programming model similar to ours to automate consistency in presence of multi-threading, respectively, of a distributed execution setting. However, they do not support a hybrid consistency model. Drechsler et al. [15] enable strong consistency (serializability) only, while Mogk et al. [37], ,38] support only eventual consistency.

7 CONCLUSION AND FUTURE WORK

In this article, we proposed LoRe, a language for local-first software with verified safety guarantees. *LoRe* combines the declarative data flow of reactive programming with static analysis and verification techniques to precisely determine concurrent interactions that could violate programmer-specified safety properties. We presented a formal definition of the programming model and a modular verification that detects concurrent executions that may violate application invariants. In case of invariant violation due to concurrent execution, LoRe automatically enforces the necessary amount of coordination. LoRe's verifying compiler translates LoRe programs to Viper [39] for automated verification and to Scala for the application logic including synthesized synchronization to guarantee the specified safety invariants. An evaluation of LoRe's programming model in two case studies confirms that it facilitates the development of safe local-first applications and enables efficient and modular automated reasoning about an application's safety properties. Our evaluation shows that verification times are acceptable and that the verification effort required from developers is reasonable.

In the future, it would be desirable to integrate existing libraries of verified CRDTs [17] or even solutions that allow *ad hoc* verification of CRDT-like data types [29, 41]. This would enable us to support a wider range of data types or even allow programmers to use custom distributed data types, which can be verified to be eventually consistent. Furthermore, our current data-flow analysis is limited to static data-flow graphs. While static reasoning about dynamic graphs is impossible in the general case, most applications make systematic use of dynamic dependencies, and we believe it would be feasible to support common cases.

REFERENCES

[1] Deepthi Devaki Akkoorath, Alejandro Z. Tomsic, Manuel Bravo, Zhongmiao Li, Tyler Crain, Annette Bieniusa, Nuno Preguiça, and Marc Shapiro. 2016. Cure: Strong semantics meets high availability and Low Latency. In *Proceedings of the IEEE 36th International Conference on Distributed Computing Systems (ICDCS'16)*. DOI: https://doi.org/10.1109/ICDCS.2016.98

[2] Peter Alvaro, Neil Conway, Joseph M. Hellerstein, and David Maier. 2014. Blazes: Coordination analysis for distributed programs. In *Proceedings of the IEEE 30th International Conference on Data Engineering*. IEEE. DOI: https://doi.org/10.1109/ICDE.2014.6816639

[3] Peter Alvaro, Neil Conway, Joseph M. Hellerstein, and William R. Marczak. 2011. Consistency analysis in Bloom: A calm and collected approach. In *Proceedings of the Conference on Innovative Data Systems Research (CIDR'11)*. Citeseer. Retrieved from http://cidrdb.org/cidr2011/Papers/CIDR11_Paper35.pdf

[4] Automerge contributors. 2023. Automerge: Build Local-First Software. Retrieved from https://automerge.org/. Accessed on 2023-04-19.

[5] Peter Bailis, Alan Fekete, Michael J. Franklin, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. 2014. Coordination avoidance in database systems. *Proc. VLDB Endow.* 8, 3 (Nov. 2014), 185–196. DOI: https://doi.org/10.14778/2735508.2735509

[6] Valter Balegas, Sérgio Duarte, Carla Ferreira, Rodrigo Rodrigues, and Nuno Preguiça. 2018. IPA: Invariant-preserving applications for weakly consistent replicated databases. *Proc. VLDB Endow.* 12, 4 (Dec. 2018), 404–418. DOI: https://doi.org/10.14778/3297753.3297760

[7] Valter Balegas, Sérgio Duarte, Carla Ferreira, Rodrigo Rodrigues, Nuno Preguiça, Mahsa Najafzadeh, and Marc Shapiro. 2015. Putting consistency back into eventual consistency. In *Proceedings of the 10th European Conference on Computer Systems (EuroSys'15)*. 1–16. DOI: https://doi.org/10.1145/2741948.2741972

[8] Valter Balegas, Sérgio Duarte, Carla Ferreira, Rodrigo Rodrigues, Nuno Preguiça, Mahsa Najafzadeh, and Marc Shapiro. 2015. Towards fast invariant preservation in geo-replicated systems. *ACM SIGOPS Operat. Syst. Rev.* 49, 1 (Jan. 2015), 121–125. DOI: https://doi.org/10.1145/2723872.2723889

[9] Mike Barnett, Bor-Yuh Evan Chang, Robert DeLine, Bart Jacobs, and K. Rustan M. Leino. 2006. Boogie: A modular reusable verifier for object-oriented programs. In *Formal Methods for Components and Objects*, Frank S. de Boer, Marcello M. Bonsangue, Susanne Graf, and Willem-Paul de Roever (Eds.). Springer, Berlin, 364–387.

[10] Eric A. Brewer. 2012. CAP twelve years later: How the "rules" have changed. *Computer* 45, 2 (2012), 23–29. DOI: https://doi.org/10.1109/MC.2012.37

[11] Kevin De Porre, Carla Ferreira, and Elisa Gonzalez Boix. 2023. VeriFx: Correct replicated data types for the masses. In *Proceedings of the 37th European Conference on Object-Oriented Programming (ECOOP'23) (Leibniz International Proceedings in Informatics (LIPIcs))*, Karim Ali and Guido Salvaneschi (Eds.), Vol. 263. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 9:1–9:45. DOI: https://doi.org/10.4230/LIPIcs.ECOOP.2023.9

[12] Kevin De Porre, Carla Ferreira, Nuno M. Preguiça, and Elisa Gonzalez Boix. 2021. ECROs: Building global scale systems from sequential code. *Proc. ACM Program. Lang.* 5, OOPSLA (2021), 1–30. DOI: https://doi.org/10.1145/3485484

[13] Kevin De Porre, Florian Myter, Christophe De Troyer, Christophe Scholliers, Wolfgang De Meuter, and Elisa Gonzalez Boix. 2019. Putting order in strong eventual consistency. In *Distributed Applications and Interoperable Systems*. Springer International Publishing, Cham, 36–56. DOI: https://doi.org/10.1007/978-3-030-22496-7_3

[14] Moritz Dinser. 2021. *An Empirical Study on Reactive Programming*. Master's thesis. Technische Universität Darmstadt. Retrieved from http://tubama.ulb.tu-darmstadt.de/id/eprint/30079

[15] Joscha Drechsler, Ragnar Mogk, Guido Salvaneschi, and Mira Mezini. 2018. Thread-safe reactive programming. *Proc. ACM Program. Lang.* 2, OOPSLA (Oct. 2018). DOI: https://doi.org/10.1145/3276477

[16] Seph Gentle. 2021. 5000x faster CRDTs: An Adventure in Optimization. Retrieved from https://josephg.com/blog/crdts-go-brrr/

[17] Victor B. F. Gomes, Martin Kleppmann, Dominic P. Mulligan, and Alastair R. Beresford. 2017. Verifying strong eventual consistency in distributed systems. *Proc. ACM Program. Lang.* 1, OOPSLA (Oct. 2017), 109:1–109:28. DOI: https://doi.org/10.1145/3133933

[18] Alexey Gotsman, Hongseok Yang, Carla Ferreira, Mahsa Najafzadeh, and Marc Shapiro. 2016. "Cause I'm strong enough": Reasoning about consistency choices in distributed systems. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. ACM, New York, NY, 371–384. DOI: https://doi.org/10.1145/2837614.2837625

[19] Joseph M. Hellerstein and Peter Alvaro. 2019. Keeping CALM: When distributed consistency is easy. Retrieved from http://arxiv.org/abs/1901.01930

[20] Farzin Houshmand and Mohsen Lesani. 2019. Hamsaz: Replication coordination analysis and synthesis. *Proc. ACM Program. Lang.* 3, POPL (Jan. 2019), 74:1–74:32. DOI: https://doi.org/10.1145/3290387

[21] Meta Platforms Inc. 2023. React: The Library for Web and Native User Interfaces. Retrieved from https://reactjs.org/

[22] Gowtham Kaki, Kapil Earanky, K. C. Sivaramakrishnan, and Suresh Jagannathan. 2018. Safe replication through bounded concurrency verification. *Proc. ACM Program. Lang.* 2, OOPSLA (Oct. 2018). DOI: https://doi.org/10.1145/3276534

[23] Gowtham Kaki, Swarn Priya, K. C. Sivaramakrishnan, and Suresh Jagannathan. 2019. Mergeable replicated data types. *Proc. ACM Program. Lang.* 3, OOPSLA (Oct. 2019), 154:1–154:29. DOI: https://doi.org/10.1145/3360580

[24] Martin Kleppmann. 2017. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly. Retrieved from https://dataintensive.net/

[25] Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. 2019. Local-first software: You own your data, in spite of the cloud. In *Proceedings of the ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! 2019)*. ACM, New York, NY. DOI: https://doi.org/10.1145/3359591.3359737

[26] Rusty Klophaus. 2010. Riak core: Building distributed applications without shared state. In *ACM SIGPLAN Commercial Users of Functional Programming*. ACM, New York, NY.

[27] Mirko Köhler, Nafise Eskandani, Pascal Weisenburger, Alessandro Margara, and Guido Salvaneschi. 2020. Rethinking safe consistency in distributed object-oriented programming. *Proc. ACM Program. Lang.* 4, OOPSLA (2020), 188:1–188:30. DOI: https://doi.org/10.1145/3428256

[28] Derrick G. Kourie and Bruce W. Watson. 2012. *The Correctness-by-Construction Approach to Programming*. Springer, Berlin. DOI: https://doi.org/10.1007/978-3-642-27919-5

[29] Shadaj Laddad, Conor Power, Mae Milano, Alvin Cheung, and Joseph M. Hellerstein. 2022. Katara: Synthesizing crdts with verified lifting. *Proc. ACM Program. Lang.* 6, OOPSLA2 (Oct. 2022), 173:1349–173:1377. DOI: https://doi.org/10.1145/3563336

[30] K. Rustan M. Leino. 2010. Dafny: An automatic program verifier for functional correctness. In *Logic for Programming, Artificial Intelligence, and Reasoning (Lecture Notes in Computer Science)*, Edmund M. Clarke and Andrei Voronkov (Eds.). Springer, Berlin, 348–370. DOI: https://doi.org/10.1007/978-3-642-17511-4_20

[31] Nicholas V. Lewchenko, Arjun Radhakrishna, Akash Gaonkar, and Pavol Černý. 2019. Sequential programming for replicated data stores. *Proc. ACM Program. Lang.* 3, ICFP (July 2019), 106:1–106:28. DOI: https://doi.org/10.1145/3341710

[32] Cheng Li, João Leitão, Allen Clement, Nuno M. Preguiça, Rodrigo Rodrigues, and Viktor Vafeiadis. 2014. Automating the choice of consistency levels in replicated systems. In *Proceedings of the USENIX Annual Technical Conference (USENIX ATC'14)*. USENIX Association, Philadelphia, PA. Retrieved from https://www.usenix.org/conference/atc14/technical-sessions/presentation/li_cheng_2

[33] Cheng Li, Daniel Porto, Allen Clement, Johannes Gehrke, Nuno Preguic, and Rodrigo Rodrigues. 2012. Making geo-replicated systems fast as possible, consistent when necessary. In *Proceedings of the 10th USENIX Conference on Operating Systems Design and Implementation (OSDI'12)*. 265–278. Retrieved from https://www.usenix.org/conference/osdi12/technical-sessions/presentation/li

[34] Prince Mahajan, Lorenzo Alvisi, Mike Dahlin, et al. 2011. Consistency, availability, and convergence. *University of Texas at Austin Tech Report*. Retrieved from https://www.cs.utexas.edu/users/dahlin/papers/cac-tr.pdf

[35] Christopher Meiklejohn and Peter Van Roy. 2015. Lasp: A language for distributed, coordination-free programming. In *Proceedings of the 17th International Symposium on Principles and Practice of Declarative Programming*. ACM, New York, NY, 184–195. DOI: https://doi.org/10.1145/2790449.2790525

[36] Mae Milano and Andrew C. Myers. 2018. MixT: A language for mixing consistency in geodistributed transactions. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, New York, NY. DOI: https://doi.org/10.1145/3192366.3192375

[37] Ragnar Mogk, Lars Baumgärtner, Guido Salvaneschi, Bernd Freisleben, and Mira Mezini. 2018. Fault-tolerant distributed reactive programming. In *Proceedings of the 32nd European Conference on Object-Oriented Programming (ECOOP'18)*, Vol. 109. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany.

[38] Ragnar Mogk, Joscha Drechsler, Guido Salvaneschi, and Mira Mezini. 2019. A fault-tolerant programming model for distributed interactive applications. *Proc. ACM Program. Lang.* 3, OOPSLA (Oct. 2019), 144:1–144:29. DOI: https://doi.org/10.1145/3360570

[39] P. Müller, M. Schwerhoff, and A. J. Summers. 2016. Viper: A verification infrastructure for permission-based reasoning. In *Verification, Model Checking, and Abstract Interpretation*, Vol. 9583. Springer-Verlag, Berlin. DOI: https://doi.org/10.1007/978-3-662-49122-5_2

[40] Florian Myter, Christophe Scholliers, and Wolfgang De Meuter. 2018. A capable distributed programming model. In *Proceedings of the ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!'18)*. ACM, New York, NY. DOI: https://doi.org/10.1145/3276954.3276957

[41] Sreeja S. Nair, Gustavo Petri, and Marc Shapiro. 2020. Proving the safety of highly-available distributed objects. In *Programming Languages and Systems*. Springer International Publishing, Cham, 544–571. DOI: https://doi.org/10.1007/978-3-030-44914-8_20

[42] Petru Nicolaescu, Kevin Jahns, Michael Derntl, and Ralf Klamma. 2016. Near real-time peer-to-peer shared editing on extensible data types. In *Proceedings of the 19th International Conference on Supporting Group Work*. ACM. DOI: https://doi.org/10.1145/2957276.2957310

[43] David J. Pearce and Lindsay Groves. 2013. Whiley: A platform for research in software verification. In *Software Language Engineering (Lecture Notes in Computer Science)*, Martin Erwig, Richard F. Paige, and Eric Van Wyk (Eds.). Springer International Publishing, Cham, 238–248. DOI: https://doi.org/10.1007/978-3-319-02654-1_13

[44] Nuno Preguiça. 2018. Conflict-free replicated data types: An overview. Retrieved from https://arxiv.org/abs/1806.10254. DOI: https://doi.org/10.48550/ARXIV.1806.10254

[45] Guido Salvaneschi, Sebastian Proksch, Sven Amann, Sarah Nadi, and Mira Mezini. 2017. On the positive effect of reactive programming on software comprehension: An empirical study. *IEEE Trans. Softw. Eng.* 43, 12 (2017). DOI: https://doi.org/10.1109/TSE.2017.2655524

[46] Marc Shapiro, Annette Bieniusa, Nuno M. Preguiça, Valter Balegas, and Christopher Meiklejohn. 2018. Just-right consistency: Reconciling availability and safety. Retrieved from http://arxiv.org/abs/1801.06340

[47] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. 2011. *A Comprehensive Study of Convergent and Commutative Replicated Data Types*. Research Report RR-7506. Inria–Centre Paris-Rocquencourt; INRIA. 50 pages. Retrieved from https://hal.inria.fr/inria-00555588

[48] K. C. Sivaramakrishnan, Gowtham Kaki, and Suresh Jagannathan. 2015. Declarative programming over eventually consistent data stores. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI'15)*. ACM, New York, NY, 413–424. DOI: https://doi.org/10.1145/2737924.2737981

[49] TPC. 2021. TPC-C Specification 5.11.0. Retrieved from http://tpc.org/TPC_Documents_Current_Versions/pdf/tpc-c_v5.11.0.pdf

[50] Viper. 2021. Viperproject/Silicon Github Repository. *Viper Project*. Retrieved from https://github.com/viperproject/silicon

[51] Matthew Weidner, Heather Miller, Huairui Qi, Maxime Kjaer, Ria Pradeep, Benito Geordie, and Christopher Meiklejohn. 2022. Collabs: Composable Collaborative Data Structures. Retrieved from 2212.02618

[52] Michael Whittaker and Joseph M. Hellerstein. 2018. Interactive checks for coordination avoidance. *Proc. VLDB Endow.* 12, 1 (Sept. 2018), 14–27. DOI: https://doi.org/10.14778/3275536.3275538

[53] George Zakhour, Pascal Weisenburger, and Guido Salvaneschi. 2023. Type-checking CRDT convergence. *Proc. ACM Program. Lang.* 7, PLDI (June 2023), 162:1365–162:1388. DOI: https://doi.org/10.1145/3591276

Received 1 March 2023; revised 21 September 2023; accepted 3 November 2023